



Computer Architecture Spring Semester 2025-2026

2nd Lab Assignment (50% of labs' grade)

Group assignment in groups of maximum three (3) students (therefore 1-3 students will be allowed per group)

Deadline: 27/5/2026, 23:59

In this lab, you will have to construct a RISC pipelined CPU. You have constructed several of the parts that your CPU will use in previous labs already. In this lab, you will construct the remaining parts, and by using these and the previous ones, you will assemble the requested processor. If you don't have fully functional components from the previous assignment, you can rebuild them in any preferred way (structural or behavioral; ALU not necessarily implemented in slices of 1bit). The description of the components can be found in the first assignment.

The components you need to implement in this assignment are the following:

Pipeline Registers

Register IF_ID

It takes as input the PC (from the PC Register) and the Instruction (from the Input), as well as the clock and mode (flush/enable). It returns the PC (i) to the ALU (which uses it via a 2-to-1 multiplexer if the instruction is MFPC) and (ii) back to the PC Register. If the enable is 1, then the PC goes to the next instruction (+2 -16-bit), while if it's flush, the contents of both the PC and outInstruction are set to zero.

Register ID_EX

It receives input from the Controller (the type of instruction), the register file, and the SignExtender. It routes the output to both the ALU, for example, if it's an R-Type instruction, and to the EX_MEM Register, for example, if it's LW/SW or PrintDigit instruction (output to the screen).

Register EX_MEM

It takes as input PrintDigit, ReadDigit, WriteEnable, isLW from the ID_EX Register, the clock from the input, the Result from the ALU, and RegAD and R2Reg from the ID_EX Register. It assigns the inputs to the output on the rising edge. RegAD_EXMEM goes to the Forwarder and to the Register File, and the Result goes both to the output and back to the input.

Register MEM_WB

It takes as input the result (either from the keyboard -keyData- if isReadDigit of the controller is true) or from the memory -fromData- if the isLW of the controller is 1, as well as the address of the register (RegAD) and returns the same prices on the rising edge. Assigns values to the writeData and writeAD signals, which go to RegisterFile, to the respective write ports.

Forwarding unit

It controls the addresses (of the register file) where a result needs to be written, but are still in the Memory or Write Back stages and their data needs to be read in parallel. If the register we read is the same as the register in one of these stages, we need to perform forwarding.

Otherwise, we read the old data. Obviously, when referring to register 0, no writing occurs, so forwarding should not be done.

Selector

Depending on the decision of the Forwarding Unit, it passes the appropriate value from the register file, memory, or WriteBack to the ALU as the first parameter. It is also used for both inputs of the ALU.

Hazard Unit

The hazard unit decides whether an instruction should be flushed. An instruction is flushed when a branch or jump instruction has been issued, causing the current instruction to be canceled. It also determines whether the next instruction should be flushed. Finally, it decides which signal should be loaded into the program counter.

Trap Unit

The trap unit monitors the opcode of each instruction. Upon receiving the End Of Running command, it forces the system into a continuous flush of instructions and freezes the program counter. This prevents the execution of invalid instructions (dummy instructions).

JRSelector

The JRSelector is a multiplexer that selects the jump address based on the JROpcode received from the Hazard Unit and passes it to the PC. If the JROpcode has the value 00, it passes the PCP2ADD as calculated by the IF_ID Register (+2 - next address). If it has the value 01, it passes the JumpAD; if it has the value 10, it passes the BranchAD. In any other case, it passes the PCP2AD.

Program Counter (PC) Register

The PC Register sends the execution address of the current instruction or jump or branch to the IF_ID register (which in turn calculates the next instruction (+2) and sends it to the ALU) as well as to the output (instructionAD).

ALU Control (alu_control)

ALU Control decides which operation the ALU should perform. In the case where the operation code is "0000", indicating an R-type instruction, the appropriate operation is selected based on the input function. In all other cases, the operation is exclusively determined by the input operation code.

Command	Description	Operation	Function	Oper. code
ADD	Addition	$O = I1 + I2$	000	0000
SUB	Subtraction	$O = I1 - I2$	001	0000
AND	AND (bit by bit)	$O = I1 \text{ AND } I2$	010	0000
OR	OR (bit by bit)	$O = I1 \text{ OR } I2$	011	0000
GEQ	Greater Equal	$O = (I1 \geq 0)$	100	0000
NOT	Not (bit by bit)	$O = (I1 == 0)$	101	0000
XOR	Exclusive OR (bit by bit)	$O = I1 \text{ XOR } I2$	110	0000
NOR	Negative OR (bit by bit)	$O = I1 \text{ NOR } I2$	111	0000

Register Control

Controls the execution process of instructions and generates all control signals. Specifically, it decides on the following. If the instruction needs to be flushed, all the above values are set to zero, so in such cases, the control will output 0.

- Whether the instruction is MFPC (move from PC)
- Whether the instruction is a Jump
- Whether the instruction is a digit read

- Whether the instruction is a print to screen
- The register where the information/result will be stored
- Whether the instruction is of type R
- Whether the instruction is a save word
- Whether the instruction is a load word
- Whether the instruction is a branch
- Whether the instruction is Jump Register

Processor

You should use the circuits you have created to assemble the CPU and perform the simulation in Quartus. The RTL diagram of the processor is shown below. You need to provide **specific examples of operations** that showcase the functionality of your processor implementation:

\$000 = 0000000000000000	As initialized
\$001 = NOT(\$000) = 1111111111111111	Operation 101
\$010 = GEQ(\$000) = 1000000000000000	Operation 100
\$011 = XOR(\$001, \$010) = 0111111111111111	Operation 110
\$100 = AND(\$010, \$011) = 0000000000000000	Operation 010
\$101 = OR(\$100, \$011) = 0111111111111111	Operation 011
\$110 = NOR(\$100, \$011) = 1000000000000000	Operation 111

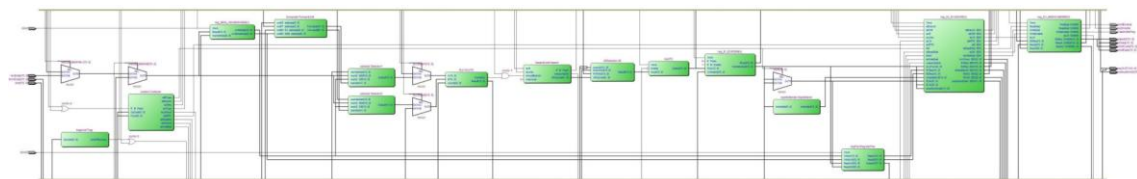


Figure 1 – RTL diagram for MIPS processor

Instructions format

Basic R-Type instructions format:

Operation (4 bits) + RS (3 bits) + RT (3 bits) + RD (3 bits) + ALU function (3 bits)

Basic memory instructions format:

Operation (4 bits) + RS (3 bits) + Offset (6 bits) + Write Register (3 bits)

Basic J-Type instructions format:

Operation (4 bits) + Offset (12 bits)

Submission instructions

For all questions you are requested to execute functional simulations to verify that your components' behavior is appropriate. You will upload the respective VHDL files, RTL diagrams of the circuits and functional simulation screenshots, which you will also have to demonstrate during the oral exams, so we can verify that your solutions are correct. You must know **how to execute** your project during the oral exams.

Only one student from each group should upload the assignment named using the following naming convention: assignment2_StudentID1_ID2_ID3.zip. **The file should be .zip (no other OS-specific compression formats like .7z). Do not compress the entire Quartus Lite project rather than only what is requested above.**